



# SuperDrive

by Tom Smith for Nexstar Media Inc.  
Current Version: 1.0.1  
For Use With Ross Overdrive

## Introduction

SuperDrive is a program written in Python 3.11.5 that assists directors and technical directors by automatically playing through lower third banners (supers) in an Overdrive environment in a logical fashion. Because Overdrive considers supers to be timeline events, the operator must cue and play them in order to bring them to air. The trouble with this is that it can lead to the operator either becoming behind because they forgot to advance to the new story super, or habitually double punching when no super is available and getting ahead of script. Unfortunately, Ross offers no native solution to allow the automatic playing of supers. SuperDrive works by constantly monitoring what's in program and preview. If a super is in preview but not in program, it emulates a key press to advance the timeline. Because this logic is implemented, the first super on any given line will always play up, but further supers on the line will be played through by the operator.

## Setup

Because it works by emulating a keyboard press, SuperDrive needs to be run on the Overdrive client machine. There is no install wizard. Simply copy the program folder to the hard drive and launch the program.

SuperDrive has the following configurable options:

| Option                     | Description                                                                                                                     | Default      |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------|
| <b>Overdrive IP</b>        | IP Address of Active Overdrive server                                                                                           | 10.10.78.10  |
| <b>Overdrive TCP Port</b>  | Port being used by Overdrive server for FloorDirectorAPI, configurable in Overdrive server                                      | 8760         |
| <b>SuperDrive TCP Port</b> | Port used by SuperDrive to listen for commands from OverDrive CC's                                                              | 8888         |
| <b>Super Keyword</b>       | The name given to the template used by supers in Overdrive, used by SuperDrive to determine whether or not something is a super | XPN 1 MOS CG |

| Option             | Description                                                                                 | Default |
|--------------------|---------------------------------------------------------------------------------------------|---------|
| <b>Advance Key</b> | The keyboard hot key used to advance the rundown in RundownControl on the Overdrive client. | space   |

These settings<sup>1</sup> are saved when changed or when SuperDrive closes to settings.pkl. They'll be automatically loaded on launch. If the file is deleted, a fresh one will be built on the next launch.

## Usage

If properly configured, SuperDrive only needs to be open and running in the background for the supers to auto advance. A status bar at the bottom of the window indicates whether or not SuperDrive is active and if a super is being played. If necessary, the program can be deactivated from the File menu and reactivated as appropriate.

SuperDrive has a simple user interface that consists of two tables displaying information as it arrives in preview and program. The bottom of the window holds a status bar which shows if the program is active and if a super is being played. The UI exists mainly as a confidence feed for the operator to confirm that data is being actively received.

## Underlying Processes

SuperDrive works by piggybacking on Ross' FloorDirectorAPI, the documentation for which is at the end of this document. This API is intended for providing the floor director or other crew members with a display of what's in program and what's in preview. SuperDrive connects to the API via TCP/IP and continually requests the latest data. If the data has changed since the previous request, SuperDrive then parses the data to determine if program/preview are supers. An item is deemed to be a super if the template of the item matches the specified Super Keyword. For this reason, SuperDrive will only work in an environment where all supers utilize the same template, I.E. all supers come out of CG1. SuperDrive then employs a basic if/then statement to determine whether or not a super should be played: If preview is a super and program is not, SuperDrive will then emulate a key press to advance the rundown in Overdrive.

---

<sup>1</sup> Although the setting is present, the SuperDrive port feature is not currently used. The feature will be enabled in a future version.

# Source Code

---

## core.py

```
# -*- coding: utf-8 -*-
"""
SuperDrive 0.1
Created on Thu Aug 22 06:17:35 2024

@author: TOSmith
"""

import socket, json, keyboard, time, threading

class OnAir(object):
    def __init__(self, parent, _index, slug, shot_name, template_name,
transition_name):
        self.parent = parent
        self._index = _index
        self.slug = slug
        self.shot_name = shot_name
        self.template_name = template_name
        self.transition_name = transition_name
        self.is_super = self.super_check(self.template_name)

    def super_check(self, template_name):
        if template_name == self.parent.super_keyword:
            return True
        return False

class Prepared(object):
    def __init__(self, parent, _index, slug, shot_name, template_name,
transition_name):
        self.parent = parent
        self._index = _index
        self.slug = slug
        self.shot_name = shot_name
        self.template_name = template_name
        self.transition_name = transition_name
        self.is_super = self.super_check(self.template_name)

    def super_check(self, template_name):
        if template_name == "XPN 1 MOS CG":
            return True
        return False

class Listener(threading.Thread):
    def __init__(self, parent, connection_info: tuple, super_keyword: str,
advance_key: str):
        threading.Thread.__init__(self)
        self.parent = parent
        self.OverDrive_IP = connection_info[0]
        self.OverDrive_port = connection_info[1]
        self.request = {'protocolVersion': '1',
                        'endpoint': 'shots',
```

```

        'correlationId': '0'
    }

    self.listening = False
    self.previous_data = None
    self.super_keyword = super_keyword
    self.advance_key = advance_key

def run(self):
    self.listening = True
    self.parent.statusbar.SetStatusText("SuperDrive is active.",0)
    self.prepared_olv = []
    self.on_air_olv = []
    self.parent.set_columns()
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        try:
            s.connect((self.OverDrive_IP, self.OverDrive_port))

            while self.listening:
                # Send the request
                s.sendall(bytearray(json.dumps(self.request), 'UTF-8'))

                # Listen for incoming data
                data = s.recv(1024)

                if not data:
                    print("Connection closed by the server.")
                    break # Exit the loop if the connection is closed

            try:
                # Convert the received data to JSON
                json_data = json.loads(data.decode('UTF-8'))

                # Check if the "shots" key is in the JSON
                if 'shots' in json_data:
                    current_data = json_data['shots']

                    # Compare with the previous data and print if
different
                    if current_data != self.previous_data:
                        print(current_data)
                        on_air =
OnAir(self, _index=str(current_data[0]['index']),
slug=str(current_data[0]
['slug']),
shot_name=str(current_data[0]
['shotName']),
template_name=str(current_data[0]['templateName']),
transition_name=str(current_data[0]['transitionName']))
                        self.on_air_olv.append(on_air)
                        prepared =
Prepared(self, _index=str(current_data[1]['index']),
slug=str(current_data[1]
['slug']),
shot_name=str(current_data[1]
['shotName']),

```

```

template_name=str(current_data[1]['templateName']),

transition_name=str(current_data[1]['transitionName']))
        self.prepared_olv.append(prepared)
        self.previous_data = current_data # Update
the previous data

self.parent.olv_preview.SetObjects(self.prepared_olv)

self.parent.olv_program.SetObjects(self.on_air_olv)
        if prepared.is super and not on_air.is super:

keyboard.press_and_release(self.advance_key)
        print('Playing super.')

self.parent.statusbar.SetStatusText('Playing super.',1)
        else:
            self.parent.statusbar.SetStatusText('',1)

        except json.JSONDecodeError:
            pass

        self.parent.statusbar.SetStatusText("SuperDrive is not
active.",0)

        except socket.error as e:
            print(f"Socket error: {e}")

            s.close()
            print('Socket closed.')

class Receiver(threading.Thread):
    def __init__(self, parent):
        super().__init__()
        self.parent = parent
        self.host = socket.gethostbyname(socket.gethostname())
        self.port = self.parent.superdrive_port
        self.running = True

    def run(self):
        # Set up the socket
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
server_socket:
            server_socket.bind((self.host, self.port))
            server_socket.listen(5)
            print(f"Listening for incoming data on {self.host}:
{self.port}...")

            while self.running:
                client_socket, addr = server_socket.accept()
                with client_socket:
                    print(f"Connection established with {addr}")
                    while self.running:
                        data = client_socket.recv(1024) # Adjust buffer size
as needed

                        data = data.decode('utf-8')
                        print(data)

```

```
        if not data:
            break
        if data == 'START':
            self.parent.on_remote_start()
        elif data == 'STOP':
            self.parent.overdrive.listening = False

def stop(self):
    self.running = False
```

---

## main.pyw

```
# -*- coding: utf-8 -*-
"""
SuperDrive GUI
Created on Thu Aug 22 08:15:40 2024

@author: TOSmith
"""

import wx #GUI
from core import Listener #Backend with FloorDirectorAPI
from ObjectListView import ObjectListView, ColumnDefn #Data Presentation
import keyboard, pickle, os #Stroke capture, data persistence, file checking
import webbrowser #Open documentation

class MainFrame(wx.Frame):
    def
__init__(self, overdrive_connection, superdrive_port, super_keyword, advance_key)
:
    super().__init__(parent=None, title="SuperDrive")
    self.title = "Super Drive 1.0.1"
    self.SetTitle(self.title)
    self.SetIcon(wx.Icon('img/icon.ico'))
    self.overdrive_connection = overdrive_connection
    self.superdrive_port = superdrive_port
    self.super_keyword = super_keyword
    self.advance_key = advance_key
    self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
    #self.receiver = Receiver(self)
    self.Bind(wx.EVT_CLOSE, self.on_close)
    '''Init Layout'''
    self.panel_main = wx.Panel(self)
    self.sizer_main = wx.FlexGridSizer(2,2,0,0)
    self.sizer_main.AddGrowableRow(1,1)
    self.sizer_main.AddGrowableCol(0,1)
    self.sizer_main.AddGrowableCol(1,1)
    self.statusbar = self.CreateStatusBar(2)

    '''Control Widgets'''
    self.label_control = wx.StaticText(self.panel_main, label="")

    '''Preview Widgets'''
    self.label_preview = wx.StaticText(self.panel_main, label="Prepared")
    self.olv_preview = ObjectListView(self.panel_main, wx.ID_ANY,
style=wx.LC_REPORT|wx.SUNKEN_BORDER|wx.EXPAND)

    '''Program Widgets'''
    self.label_program = wx.StaticText(self.panel_main, label="On Air")
    self.olv_program = ObjectListView(self.panel_main, wx.ID_ANY,
style=wx.LC_REPORT|wx.SUNKEN_BORDER|wx.EXPAND)

    '''Filling Sizers'''
    self.sizer_main.AddMany([(self.label_preview, 1, wx.ALL|wx.CENTER),
                             (self.label_program, 1, wx.ALL|wx.CENTER),
                             (self.olv_preview, 1, wx.ALL|wx.EXPAND),
```

```

        (self.olv_program, 1, wx.ALL|wx.EXPAND)])
self.panel_main.SetSizerAndFit(self.sizer_main)
self.SetInitialSize(self.GetBestSize())
self.SetMinSize((450,100))
self.Layout()
self.overdrive.start()
#self.receiver.start()
self.create_menu()
self.Show()

def set_columns(self):
    self.col_index = ColumnDefn("Page", "left", -1, "_index",
isSpaceFilling=True)
    self.col_slug = ColumnDefn("Slug", "left", -1, "slug",
isSpaceFilling=True)
    self.col_shot = ColumnDefn("Shot", "left", -1, "shot_name",
isSpaceFilling=True)
    self.col_template = ColumnDefn("Template", "left", -1,
"template_name", isSpaceFilling=True)
    self.col_transition = ColumnDefn("Transition", "left", -1,
"transition_name", isSpaceFilling=True)
    self.olv_preview.SetColumns([self.col_index,
                                self.col_slug,
                                self.col_shot,
                                self.col_template,
                                self.col_transition])
    self.olv_program.SetColumns([self.col_index,
                                self.col_slug,
                                self.col_shot,
                                self.col_template,
                                self.col_transition])

    try:
        self.olv_preview.SetObjects(self.overdrive.prepared_olv)
    except Exception as e:
        print(e)

    try:
        self.olv_program.SetObjects(self.overdrive.on_air_olv)
    except Exception as e:
        print(e)

def on_close(self, event):
    dlg = wx.MessageDialog(self, 'Are you sure you want to quit
SuperDrive?', 'Quit?', style=wx.YES_NO)
    result = dlg.ShowModal()
    if result == wx.ID_YES:
        self.overdrive.listening = False
        if hasattr(self, 'receiver'):
            self.receiver.receiving = False
        with open('settings.pkl', 'wb') as file:
            settings = [self.overdrive_connection,
                        self.superdrive_port,
                        self.super_keyword,
                        self.advance_key]
            pickle.dump(settings, file)
            file.close()
        self.Destroy()

def create_menu(self):

```



```

menu_bar = wx.MenuBar()

file_menu = wx.Menu()
active = file_menu.Append(wx.ID_ANY, 'Activate/Deactivate', 'Start or
stop SuperDrive.')
self.Bind(event=wx.EVT_MENU, handler=self.on_active, source=active)
_quit = file_menu.Append(wx.ID_ANY, 'Quit', 'Quit SuperDrive.')
self.Bind(event=wx.EVT_MENU, handler=self.on_close, source=_quit)
menu_bar.Append(file_menu, '&File')

edit_menu = wx.Menu()
settings = edit_menu.Append(wx.ID_ANY, 'Settings', 'Adjust settings')
self.Bind(event=wx.EVT_MENU, handler=self.on_settings, source=settings)
menu_bar.Append(edit_menu, '&Edit')

help_menu = wx.Menu()
about_option = help_menu.Append(wx.ID_ANY, 'About', 'Information about
this program.')

self.Bind(event=wx.EVT_MENU, handler=self.on_about, source=about_option)

documentation_option = help_menu.Append(wx.ID_ANY,
'Documentation', "Opens a PDF of this program's documentation.")

self.Bind(event=wx.EVT_MENU, handler=self.on_documentation, source=documentatio
n_option)

menu_bar.Append(help_menu, '&Help')

self.SetMenuBar(menu_bar)

def on_active(self, event):
    if self.overdrive.listening:
        self.overdrive.listening = False
    else:
        self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
        self.overdrive.start()

def on_remote_start(self):
    self.overdrive.listening = False
    self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
    self.overdrive.start()

def on_settings(self, event):
    SettingsFrame(self)

def on_about(self, event):
    AboutFrame(self)

def on_documentation(self, event):
    path = os.getcwd()
    webbrowser.open(f'{path}\\documentation\\documentation.pdf')

class SettingsFrame(wx.Frame):
    def __init__(self, parent):
        super().__init__(parent=parent, title="SuperDrive Settings")

```

```

self.parent = parent
self.Bind(wx.EVT_CLOSE, self.on_close)

'''Init UI'''
self.SetTitle(f'{self.parent.title} - Settings')
self.SetIcon(wx.Icon('img/icon.ico'))
self.parent = parent
self.panel_main = wx.Panel(self)
self.sizer = wx.FlexGridSizer(6,2,10,10)
self.sizer_buttons = wx.BoxSizer(wx.HORIZONTAL)

'''Widgets'''
self.label_key = wx.StaticText(self.panel_main, label="OverDrive
Advance Key")
self.button_key = wx.ToggleButton(self.panel_main, label="Assign
Key")
self.button_key.SetLabel(self.parent.advance_key)
self.assigned = self.parent.advance_key
self.button_key.Bind(wx.EVT_TOGGLEBUTTON, self.assign_key)
self.button_key.Bind(wx.EVT_KEY_UP, self.get_key)

self.label_overdrive_ip =
wx.StaticText(self.panel_main, label="OverDrive Server IP")
self.field_overdrive_ip = wx.TextCtrl(self.panel_main)
self.field_overdrive_ip.SetValue(self.parent.overdrive.OverDrive_IP)

self.label_overdrive_port =
wx.StaticText(self.panel_main, label="OverDrive Port")
self.field_overdrive_port = wx.TextCtrl(self.panel_main)

self.field_overdrive_port.SetValue(str(self.parent.overdrive.OverDrive_port))

self.label_superdrive_port =
wx.StaticText(self.panel_main, label="SuperDrive Port")
self.field_superdrive_port = wx.TextCtrl(self.panel_main)
self.field_superdrive_port.SetValue(str(self.parent.superdrive_port))

self.label_super_keyword = wx.StaticText(self.panel_main, label="Super
Keyword")
self.field_super_keyword = wx.TextCtrl(self.panel_main)

self.field_super_keyword.SetValue(self.parent.overdrive.super_keyword)

self.button_apply = wx.Button(self.panel_main, label="Apply")
self.button_apply.Bind(wx.EVT_BUTTON, self.on_apply)
self.button_cancel = wx.Button(self.panel_main, label="Cancel")
self.button_cancel.Bind(wx.EVT_BUTTON, self.on_cancel)

'''Filling Sizers'''
self.sizer_buttons.AddMany([(self.button_apply, 1, wx.ALL|wx.CENTER),
                           (self.button_cancel, 1, wx.ALL|wx.CENTER)])
self.sizer.AddMany([(self.label_key, 1, wx.ALL|wx.CENTER),
                    (self.button_key, 1, wx.ALL|wx.CENTER),
                    (self.label_overdrive_ip, 1, wx.ALL|wx.CENTER),
                    (self.field_overdrive_ip, 1, wx.ALL|wx.CENTER),
                    (self.label_overdrive_port, 1, wx.ALL|wx.CENTER),
                    (self.field_overdrive_port, 1, wx.ALL|wx.CENTER),
                    (self.label_superdrive_port, 1, wx.ALL|wx.CENTER),

```

```

        (self.field_superdrive_port, 1, wx.ALL | wx.CENTER),
        (self.label_super_keyword, 1, wx.ALL | wx.CENTER),
        (self.field_super_keyword, 1, wx.ALL | wx.CENTER),
        (self.sizer_buttons, 1, wx.ALL | wx.CENTER) ])
self.panel_main.SetSizerAndFit(self.sizer)
self.SetInitialSize(self.GetBestSize())
self.Layout()
self.Show()

def get_key(self, event):
    if self.button_key.GetValue() == True:
        self.assigned = keyboard.read_key()
        self.button_key.SetLabel(str(self.assigned))
        self.button_key.SetValue(False)
        self.panel_main.SetFocus()
        print(f'Assigned {self.assigned} as key.')

def assign_key(self, event):
    self.panel_main.SetFocus()
    if hasattr(self, 'assigned'):
        del(self.assigned)
    self.button_key.SetValue(True)
    self.button_key.SetLabel("[[Press any key.]]")

def on_close(self, event):
    dlg = wx.MessageDialog(self, 'Quit without saving your
changes?', 'Quit?', wx.YES_NO)
    result = dlg.ShowModal()
    if result == wx.ID_YES:
        self.Destroy()

def on_cancel(self, event):
    self.Destroy()

def on_apply(self, event):
    if hasattr(self, 'assigned'):
        self.parent.overdrive.advance_key = self.assigned
    overdrive_ip = self.field_overdrive_ip.GetValue()
    overdrive_port = int(self.field_overdrive_port.GetValue())
    self.parent.overdrive_connection = (overdrive_ip, overdrive_port)
    self.parent.superdrive_port =
int(self.field_superdrive_port.GetValue())
    self.parent.super_keyword = self.field_super_keyword.GetValue()
    if self.parent.overdrive.listening:
        self.parent.overdrive.listening = False
    self.parent.overdrive = Listener(self.parent,
self.parent.overdrive_connection, self.parent.super_keyword,
self.parent.advance_key)
    self.parent.overdrive.start()
    self.Destroy()

class AboutFrame(wx.Frame):
    def __init__(self, parent):
        super().__init__(parent=parent)
        self.parent = parent
        self.SetTitle(f'{self.parent.title} - About')
        self.SetIcon(wx.Icon('img/icon.ico'))
        self.panel_main = wx.Panel(self)

```

```

        self.sizer_main = wx.FlexGridSizer(6,1,10,10)
        self.font = wx.Font(12, wx.FONTFAMILY_MODERN, 0, 90, underline =
False, faceName="Arial Bold")
        self.logo = wx.Image('img/icon.png', wx.BITMAP_TYPE_PNG)
        self.bitmap_logo =
wx.StaticBitmap(self.panel_main,bitmap=self.logo.ConvertToBitmap())
        self.label_program_name = wx.StaticText(self.panel_main,
label=self.parent.title)
        self.label_program_name.SetFont(self.font)
        self.label_byline = wx.StaticText(self.panel_main, label="by Tom
Smith for Nexstar Media Inc.")
        self.label_email = wx.StaticText(self.panel_main,
label="thomas.smith@woodtv.com")
        self.label_phone_cell = wx.StaticText(self.panel_main, label="Cell:
(231) 343.9803")
        self.label_phone_desk = wx.StaticText(self.panel_main, label="Desk:
(616) 456.8888 Ext. 4319")

        self.sizer_main.AddMany([(self.bitmap_logo,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                (self.label_program_name,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                (self.label_byline,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                (self.label_email,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                (self.label_phone_cell,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                (self.label_phone_desk,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER)])

        self.panel_main.SetSizerAndFit(self.sizer_main)
        self.SetInitialSize(self.GetBestSize())
        self.Layout()
        self.Show()

def main():
    if os.path.isfile('settings.pkl'):
        with open('settings.pkl','rb') as file:
            overdrive_connection, superdrive_port, super_keyword, advance_key
= pickle.load(file)
            file.close()
    else:
        overdrive_connection = ('10.10.78.10', 8760)
        superdrive_port = 8888
        super_keyword = "XPN 1 MOS CG"
        advance_key = "space"
        app=[]; app = wx.App(None)
        frame = MainFrame(overdrive_connection, superdrive_port, super_keyword,
advance_key)
        app.SetTopWindow(frame)
        app.MainLoop()

if __name__ == "__main__":
    main()

```



---

## FloorDirector API Guide

Version 1.1



# Thank You for Choosing Ross

You've made a great choice. We expect you will be very happy with your purchase of Ross Technology.

Our mission is to:

1. Provide a Superior Customer Experience
  - offer the best product quality and support
2. Make Cool Practical Technology
  - develop great products that customers love

Ross has become well known for the Ross Video Code of Ethics. It guides our interactions and empowers our employees. I hope you enjoy reading it below.

If anything at all with your Ross experience does not live up to your expectations be sure to reach out to us at [solutions@rossvideo.com](mailto:solutions@rossvideo.com).



David Ross

CEO, Ross Video

[david.ross@rossvideo.com](mailto:david.ross@rossvideo.com)

## Ross Video Code of Ethics

Any company is the sum total of the people that make things happen. At Ross, our employees are a special group. Our employees truly care about doing a great job and delivering a high quality customer experience every day. This code of ethics hangs on the wall of all Ross Video locations to guide our behavior:

1. We will always act in our customers' best interest.
3. We will do our best to understand our customers' requirements.
4. We will not ship crap.
5. We will be great to work with.
6. We will do something extra for our customers, as an apology, when something big goes wrong and it's our fault.
7. We will keep our promises.
8. We will treat the competition with respect.
9. We will cooperate with and help other friendly companies.
10. We will go above and beyond in times of crisis. If there's no one to authorize the required action in times of company or customer crisis - do what you know in your heart is right. (You may rent helicopters if necessary.)





# FloorDirector API Guide

- Ross Part Number: 7900DR-010-1.1
- Release Date: June 1, 2023. Printed in Canada.
- Software Issue: 1.1

The information contained in this Guide is subject to change without notice or obligation.

## Copyright

© 2016 - 2023 Ross Video Limited. Ross® and any related marks are trademarks or registered trademarks of Ross Video Limited. All other trademarks are the property of their respective companies. PATENTS ISSUED and PENDING. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, mechanical, photocopying, recording or otherwise, without the prior written permission of Ross Video. While every precaution has been taken in the preparation of this document, Ross Video assumes no responsibility for errors or omissions. Neither is any liability assumed for damages resulting from the use of the information contained herein.

## Patents

Patent numbers 4,205,346; 5,115,314; 5,280,346; 5,561,404; 7,034,886; 7,508,455; 7,602,446; 7,834,886; 7,914,332; 8307284, 2039277; 1237518; 1127289 and other patents pending.

## Notice

The material in this manual is furnished for informational use only. It is subject to change without notice and should not be construed as commitment by Ross Video Limited. Ross Video Limited assumes no responsibility or liability for errors or inaccuracies that may appear in this manual.

## Company Address

|                               |                           |
|-------------------------------|---------------------------|
| Ross Video Limited            | Ross Video Incorporated   |
| 8 John Street                 | P.O. Box 880              |
| Iroquois, Ontario             | Ogdensburg, New York      |
| Canada, K0E 1K0               | USA 13669-0880            |
| <hr/>                         |                           |
| General Business Office:      | (+1) 613 - 652 - 4886     |
| Fax:                          | (+1) 613 - 652 - 4425     |
| Technical Support:            | (+1) 613 - 652 - 4886     |
| After Hours Emergency:        | (+1) 613 - 349 - 0006     |
| E-mail (Technical Support):   | techsupport@rossvideo.com |
| E-mail (General Information): | solutions@rossvideo.com   |
| Website:                      | http://www.rossvideo.com  |

# Contents

- Overdrive FloorDirector API Services ..... 3
  - Overview ..... 3
  - API Endpoints (Summary) ..... 3
- RESTful Services ..... 3
  - REST API PATH and Version ..... 3
  - Parameters ..... 4
  - Message Exchange ..... 4
- Socket Services ..... 4
  - Message Transport..... 4
  - Message Request ..... 4
  - Message Exchange ..... 5
- API Endpoint Definitions..... 6
  - Timers..... 6
  - Shots ..... 11
  - Info ..... 13
  - Cues ..... 16
  - Variable ..... 18
  - Subscription..... 21
  - Error Response ..... 23



# Overdrive FloorDirector API Services

## Overview

The FloorDirector API will enable users to query timing, shot, cue, and OD system information. JSON responses will be returned for supported queries.

There will be two types of services: RESTful and Socket.

- The RESTful Services will provide the access to the API through http connections.
  - > The simplest one. Easy to test. Can connect even from JavaScript.
  - > No (firewall) security system port rules needed (always port 80)
  - > The request parameters must be provided in a GET Query String.
- The [Socket Services](#) will provide the access to the API through a socket connection.
  - > Clients must implement a socket connection.
  - > Clients are responsible for keeping the connection open (OD will send responses and updates through the same socket the client opened). Therefore, clients should monitor their sockets to check the connection health.
  - > The request parameters must be provided in a JSON message.

Although the requests may differ depending on the service type (REST or Socket), the response will be always in the same JSON format described in the API Endpoint Definitions.

## API Endpoints (Summary)

The FloorDirector API will expose the following endpoints.

| Endpoint                     | Description                                                                                                                                                                          |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">timers</a>       | Gets the rundown timing information.                                                                                                                                                 |
| <a href="#">shots</a>        | Gets the OnAir/Prepared shot information.                                                                                                                                            |
| <a href="#">info</a>         | Gets the system information (Playing Rundown Name, controlling RC host, active server host, backup server host).                                                                     |
| <a href="#">cues</a>         | Gets the last 'cue' messages.                                                                                                                                                        |
| <a href="#">variable</a>     | Gets rundown variable view information (audio variable status).                                                                                                                      |
| <a href="#">subscription</a> | <b>Socket Only</b> - It subscribes/unsubscribes the current client connection for receiving autonomous timing updates. The default behavior for new connections is being subscribed. |

For more endpoint definitions see "[API Endpoint Definitions](#)".

## RESTful Services

### REST API PATH and Version

The base path will be set to `http://<odserver>/server/floordirector/api/<endpoint>`

- The version of the API (v1) will appear as a prefix to the endpoint.
  - > For example, to get "timers" info, you'd make a GET request to:  
`http://<odserver>/server/floordirector/api/v1/timers?<parameters>`

## Parameters

The request parameters must be provided in a GET Query String

For non-ASCII characters:

- Represent each character in UTF-8 (see [RFC2279](#)) as one or more bytes.
- Escape these bytes with the URI escaping mechanism (i.e., by converting each byte to %HH, where HH is the hexadecimal notation of the byte value).

Also, see:

- <https://www.w3.org/TR/html40/appendix/notes.html#non-ascii-chars>
- [https://en.wikipedia.org/wiki/Query\\_string](https://en.wikipedia.org/wiki/Query_string)

## Message Exchange

Because the RESTful API is a server-client service (HTTP), the client always has to pull data from the server.

For timing updates, the client must implement a mechanism to constantly pull data from the server (half a second should be enough).

# Socket Services

The interaction can be generated either by the client (**Request-Response**) or by the server (**Broadcast**, IFF the client has already established a socket connection).

## Message Transport

Clients must open a socket to the defined Overdrive FloorDirector port. The default TCP/IP port will be set to **8760** (it could be re-configured from the Overdrive WEB Server Configuration page). In order to prevent client applications from exhausting the server resources, there will be a limit of up to 100 open connections to the server. Besides that, to prevent a single client application from exhausting the server connection capacity, there will be a limit of up to 10 open connections per client IP address.

## Message Request

All the client requests must be sent in a single message in JSON format.

### Request JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "protocolVersion": {
      "type": "string"
    },
    "endpoint": {
      "type": "string"
    },
    "reqData": {
      "type": "string"
    },
    "correlationId": {
      "type": "string"
    }
  }
}
```

## Field Description

| Field           | Description                                                                                                                                                                         | Values                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| protocolVersion | The version of the API.                                                                                                                                                             | 1                                                                                                                        |
| endpoint        | API exposed endpoints.                                                                                                                                                              | <a href="#">timers</a> , <a href="#">shots</a> , <a href="#">info</a> , <a href="#">cues</a> , <a href="#">variables</a> |
| reqData         | Required Endpoint Parameters, query string format as follows:<br>param1=value1&param2=value2 ...                                                                                    | See <a href="#">API Endpoint Definitions</a>                                                                             |
| correlationId   | Since the server can send multiple asynchronous updates to the client, we recommend the client to include an id in the request, the response to that request will have the same id. |                                                                                                                          |

## Example

```
{
  "protocolVersion": "1",
  "endpoint": "timers",
  "reqData": "type=user&timerId=Timer1",
  "correlationId": "1000001"
}
```

## Message Exchange

To request updates to the server and receive them:

1. The client application will open a socket on the appropriate port to the OD Server (if a socket has not already been established)
2. The client will then hold the socket open
3. The client application can send the message and wait for any incoming updates on the same socket.

**Important Note:** Once a client connects and sends the first request message it will automatically receive autonomous updates (see [Subscription](#) endpoint). The default timing update frequency will be 500ms but it could be re-configured from the Overdrive WEB Server FloorDirector page.

# API Endpoint Definitions

For simplifying the examples, they show only a REST request, see [Socket Services](#) for building a socket request.

## Timers

Gets rundown timing information (timing sources).

### Parameters

| URL Parameters | Description                                                                                                  | Values                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Required |
|----------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| type           | The timing source type.                                                                                      | “static” – only the static timers.<br>“user” – only the user timers (per Show).<br>“” – empty for all.                                                                                                                                                                                                                                                                                                                                                                           | No       |
| timerId        | Limits response to only include the named timer id. If omitted, all timers will be included in the response. | Static Timer Names: <ul style="list-style-type: none"><li>• “Program Time Elapsed”</li><li>• “Shot Time Elapsed”</li><li>• “Story Time Elapsed”</li><li>• “Clip Time Elapsed (*TYPE-ID*)”</li><li>• “Clip Time Remaining (*TYPE-ID*)”</li><li>• “Clock (12 Hour)”</li><li>• “Clock (24 Hour)”</li><li>• “NRCS Estimated Duration Remaining”</li><li>• “NRCS Target Time Remaining”</li><li>• “NRCS Media Time Remaining”</li><li>• “NRCS Rundown Start Time Remaining”</li></ul> | No       |

**\*TYPE-ID\*:** Static clip timers depend on the crosspoints or video servers they are based to. You must specify an extra id to get the corresponding clip timer (as is in RC), add:

- XPT-<crosspoint>: When the Clip source is a Crosspoint.
- VS-<videoserver>: When the Clip source is a Video Server.

For example:

- To get a static clip timer from (let’s say) the crosspoint 24, you would use:
  - > ‘timerId=Clip Time Elapsed (XPT-24)’
  - > ‘timerId=Clip Time Remaining (XPT-24)’
- To get a static clip timer from a video server named BlackStormVS, you would use:
  - > ‘timerId=Clip Time Elapsed (VS- BlackStormVS)’
  - > ‘timerId=Clip Time Remaining (VS-BlackStormVS)’

If you request static timers without filters, you will get both all crosspoint-based and all videosever-based timers.

## Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        },
        "correlationId": {
          "type": "string"
        }
      }
    },
    "timers": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "timerId": {
            "type": "string"
          },
          " userTimerName ": {
            "type": "string"
          },
          "type": {
            "type": "string"
          },
          "countThrough": {
            "type": "boolean"
          },
          "direction": {
            "type": "string"
          },
          "running": {
            "type": "boolean"
          },
          "duration": {
            "type": "integer"
          },
          "startTime": {
            "type": "string"
          },
          "currentValue": {
            "type": "string"
          },
          "clipName": {
            "type": "string"
          },
          "position": {
            "type": "integer"
          },
          "manualBehaviour": {
            "type": "boolean"
          }
        }
      }
    }
  }
}
```





## Field Description

| Field             | Description                                                                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>meta</b>       | Metadata information about the target server.                                                                                                                                                               |
| • serverDate      | Format is (ISO8601): YYYY-MM-DD'Thh:mm:ss[,ddd]['Z'].                                                                                                                                                       |
| • odHost          | IP or hostname.                                                                                                                                                                                             |
| • odVersion       | If the client sent a correlationId in the request, it would be shown here in the response.                                                                                                                  |
| • correlationId   | If the client sent a correlationId in the request, it would be shown here in the response.                                                                                                                  |
| <b>timers</b>     | Timer information.                                                                                                                                                                                          |
| • timerId         | Timer Identifier.                                                                                                                                                                                           |
| • userTimerName   | For user timers: timer name defined by the user per Show.<br>For static timers: this has the same value as “timerId”.                                                                                       |
| • type            | Timer type (static, user).                                                                                                                                                                                  |
| • duration        | Source total duration (if set) in milliseconds.                                                                                                                                                             |
| • startTime       | Start time for the current timer. Format is (ISO8601): YYYY-MM-DD'Thh:mm:ss[,ddd]['Z'].                                                                                                                     |
| • countThrough    | Allow count through.                                                                                                                                                                                        |
| • direction       | Counting direction: “UP” or “DOWN” (case insensitive ).                                                                                                                                                     |
| • running         | True if this timer is running.                                                                                                                                                                              |
| • currentValue    | The current value for this timer. Calculated using: <ul style="list-style-type: none"> <li>• (Direction-up): serverDate-startTime.</li> <li>• (Direction-down): duration-(serverDate-startTime).</li> </ul> |
| • clipName        | Current clip name (when timer is a Clip Elapsed or Remained type).                                                                                                                                          |
| • position        | Current elapsed time in milliseconds.                                                                                                                                                                       |
| • manualBehaviour | True if the timer has a manual behaviour,                                                                                                                                                                   |

### Example

REST: GET http://<od>/server/floordirector/api/v1/timers?

```
{
  "meta": {
    "serverDate": "2009-04-11T14:22:07,125-0500",
    "odHost": "serverhost01",
    "odVersion": "16.3.0"
    "correlationId": "1000001"
  },
  "timers": [
    {
      "timerId": "Shot Time Elapsed",
      "userTimerName": "Shot Time Elapsed",
      "type": "static",
      "countThrough": true,
      "direction": "down",
      "running": true,
      "duration": 1000,
      "startTime": "2009-04-11T14:22:07,125-0500",
      "currentValue": "01:12",
      "clipName": "",
      "position": 7200,
      "manualBehaviour": false
    },
    {
      "timerId": "Timer01",
      "userTimerName": "BlackStormVO",
      "type": "user",
      "countThrough": false,
      "direction": "up",
      "running": true,
      "duration": 2000,
      "startTime": "2009-04-11T14:22:07,125-0500",
      "currentValue": "11:31"
      "clipName": "",
      "position": 7200,
      "manualBehaviour": false
    }
  ]
}
```

# Shots

Gets shot information (OnAir/Prepared shots).

## Parameters

| URL Parameters |  | Description                               | Values                                                                                                                 | Required |
|----------------|--|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------|
| type           |  | The shots to be included in the response. | “onair” – only the onair shot.<br>“prepared” – only the prepared shot.<br>“” – empty for both onair and prepare shots. | No       |

## Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        },
        "correlationId": {
          "type": "string"
        }
      }
    },
    "shots": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "type": {
            "type": "string"
          },
          "index": {
            "type": "string"
          },
          "slug": {
            "type": "string"
          },
          "shotName": {
            "type": "string"
          },
          "templateName": {
            "type": "string"
          },
          "transitionName": {
            "type": "string"
          }
        }
      }
    }
  }
}
```

## Field Description

| Field            | Description                                                                                |
|------------------|--------------------------------------------------------------------------------------------|
| <b>meta</b>      | Metadata information about the target server.                                              |
| • serverDate     | Format is (ISO8601): YYYY-MM-DD'T'hh:mm:ss[,ddd][Z].                                       |
| • odHost         | IP or hostname.                                                                            |
| • odVersion      | If the client sent a correlationId in the request, it would be shown here in the response. |
| • correlationId  | If the client sent a correlationId in the request, it would be shown here in the response. |
| <b>shots</b>     | Shot information.                                                                          |
| • type           | Shot type (onair or prepared).                                                             |
| • slug           | Slug.                                                                                      |
| • index          | Shot index.                                                                                |
| • shotName       | Shot name.                                                                                 |
| • templateName   | Shot template name.                                                                        |
| • transitionName | Shot transition name.                                                                      |

## Example

REST: GET http://<od>/server/floordirector/api/v1/shots?

```
{
  "meta": {
    "serverDate": "2009-04-11T14:22:07,125-0500",
    "odHost": "serverhost01",
    "odVersion": "16.3.0"
    "correlationId": "1000001"
  },
  "shots": [
    {
      "type": "onair",
      "index": "A1",
      "slug": "BREAKING NEWS",
      "shotName": "Shot CAM 101",
      "templateName": "CAM 101",
      "transitionName": "CUT"
    },
    {
      "type": "prepared",
      "index": "A2",
      "slug": "BREAK",
      "shotName": "Shot BLACK",
      "templateName": "BLACK",
      "transitionName": "WIPE"
    }
  ]
}
```

## Info

Gets system information (Playing Rundown Name, controlling RC client, active and backup servers).

### Parameters

| URL Parameters | Description | Values | Required |
|----------------|-------------|--------|----------|
| None           |             |        |          |

### Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        },
        "correlationId": {
          "type": "string"
        }
      }
    },
    "info": {
      "type": "object",
      "properties": {
        "playingRundownName": {
          "type": "string"
        },
        "controllingRcClient": {
          "type": "string"
        },
        "activeServer": {
          "type": "string"
        },
        "backupServer": {
          "type": "string"
        },
        "onAirModeEnabled": {
          "type": "boolean"
        }
      }
    },
    "devices": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "type": {
            "type": "string"
          },
          "identifier": {
            "type": "string"
          },
          "mode": {
            "type": "string"
          },
          "location": {
            "type": "string"
          }
        }
      }
    }
  }
}
```

```
}  
}  
}  
}  
}  
}  
}  
},  
"status": {  
  "type": "string"  
}
```

## Field Description

| Field                 | Description                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------|
| <b>meta</b>           | Metadata information about the target server.                                              |
| • serverDate          | Format is (ISO8601): YYYY-MM-DD'Thh:mm:ss[,ddd][['Z']].                                    |
| • odHost              | IP or hostname.                                                                            |
| • odVersion           | If the client sent a correlationId in the request, it would be shown here in the response. |
| • correlationId       | If the client sent a correlationId in the request, it would be shown here in the response. |
| <b>info</b>           |                                                                                            |
| • playingRundownName  | Name of the playing rundown.                                                               |
| • controllingRcClient | Controlling RundownControl client.                                                         |
| • activeServer        | Active Overdrive server hostname or IP address.                                            |
| • backupServer        | Backup Overdrive server hostname or IP address.                                            |
| • onAirModeEnabled    | True when the Playing RundownClient has the OnAir Mode enabled. False otherwise.           |
| • devices             | Device status information.                                                                 |
| ○ type                | Type of OverDrive device or component.                                                     |
| ○ identifier          | Name or level of the current device or component.                                          |
| ○ mode                | Current OverDrive redundant system mode of the device or component.                        |
| ○ location            | IP address or hostname or virtual location of the device or component.                     |
| ○ status              | Current status of the component.                                                           |

### Example

REST: GET http://<od>/server/floordirector/api/v1/info?

```
{
  "meta": {
    "serverDate": "2020-04-11T14:22:07,125-0500",
    "odHost": "serverhost01",
    "odVersion": "20.0.1"
    "correlationId": "1000001"
  },
  "info": {
    "playingRunDownName": "[NRCS] INEWS Morning",
    "controllingRcClient": "192.168.10.24",
    "activeServer": "srvotto0001",
    "backupServer": "srvotto0002",
    "onAirModeEnabled": true,
    "devices": [
      {
        "type": "quickturn",
        "identifier": "QT-MOS-NAME",
        "mode": "primary",
        "location": "srvottcap001 (chan1)",
        "status": "RECORDING"
      },
      {
        "type": "quickturn",
        "identifier": "QT-NON-MOS-NAME",
        "mode": "primary",
        "location": "srvottcap001 (chan2)",
        "status": "STOP"
      }
    ]
  }
}
```



## Cues

Gets the last 'cue' messages.

### Parameters

| URL Parameters | Description | Values | Required |
|----------------|-------------|--------|----------|
| None           |             |        |          |

### Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        },
        "correlationId": {
          "type": "string"
        }
      }
    },
    "cues": {
      "type": "object",
      "properties": {
        "lastOnAirMsg": {
          "type": "string"
        },
        "lastOnAirMsgDate": {
          "type": "string"
        },
        "lastPreparedMsg": {
          "type": "string"
        },
        "lastPreparedMsgDate": {
          "type": "string"
        }
      }
    }
  }
}
```

## Field Description

| Field                 | Description                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------|
| <b>meta</b>           | Metadata information about the target server.                                               |
| • serverDate          | Format is (ISO8601): YYYY-MM-DD'T'hh:mm:ss[,ddd]['Z'].                                      |
| • odHost              | IP or hostname.                                                                             |
| • odVersion           | If the client sent a correlationId in the request, it would be shown here in the response.  |
| • correlationId       | If the client sent a correlationId in the request, it would be shown here in the response.  |
| <b>cues</b>           |                                                                                             |
| • lastOnAirMsg        | The last on-air message.                                                                    |
| • lastOnAirMsgDate    | Date and time of the last on-air message in the format: YYYY-MM-DD'T'hh:mm:ss[,ddd]['Z'].   |
| • lastPreparedMsg     | The last prepared message.                                                                  |
| • lastPreparedMsgDate | Date and time of the last prepared message in the format: YYYY-MM-DD'T'hh:mm:ss[,ddd]['Z']. |

## Example

REST: GET http://<od>/server/floordirector/api/v1/cues?

```
{
  "meta": {
    "serverDate": "2009-04-11T14:22:07,125-0500",
    "odHost": "serverhost01",
    "odVersion": "16.3.0"
    "correlationId": "1000001"
  },
  "cues": {
    "lastOnAirMsg": "Prepare CAM 1!",
    "lastOnAirMsgDate": "2009-04-11T14:22:07,125-0500",
    "lastPreparedMsg": "Prepare CAM 2!",
    "lastPreparedMsgDate": "2009-04-11T14:22:07,125-0500"
  }
}
```

## Variable

Gets rundown variable view information (audio variable status).

### Parameters

| URL Parameters | Description                                                                                      | Values                                                                                            | Required |
|----------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------|
| name           | Only gets the variable status source with this name. If empty, returns the status all variables. | “variable name” – only the named variable.<br>“” – empty for the statuses of all audio variables. | No       |

### Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        }
      }
    },
    "statuses": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "variable": {
            "type": "object",
            "properties": {
              "variableId": {
                "type": "integer"
              },
              "variableName": {
                "type": "string"
              },
              "defaultSource": {
                "type": "string"
              }
            }
          },
          "next": {
            "type": "object",
            "properties": {
              "source": {
                "type": "string"
              },
              "status": {
                "type": "string"
              }
            }
          },
          "assigned": {
            "type": "object",
            "properties": {
              "source": {
                "type": "string"
              }
            }
          }
        }
      }
    }
  }
}
```

```
    },
    "status": {
      "type": "string"
    }
  },
},
"nextShow": {
  "type": "object",
  "properties": {
    "source": {
      "type": "string"
    },
    "status": {
      "type": "string"
    }
  }
},
},
"assignedShow": {
  "type": "object",
  "properties": {
    "source": {
      "type": "string"
    },
    "status": {
      "type": "string"
    }
  }
},
},
"isLocked": {
  "type": "boolean"
}
}
}
}
```

## Field Description

| Field           | Description                                                                            |
|-----------------|----------------------------------------------------------------------------------------|
| <b>meta</b>     | Metadata information about the target server.                                          |
| • server-date   | Format is YYYY-MM-DD'Thh:mm:ss[,ddd][['Z']].                                           |
| • od-host       | IP address or hostname.                                                                |
| • od-version    | Overdrive version (MAYOR).(MINOR).                                                     |
| <b>statuses</b> | Audio variable view status information.                                                |
| • variable      | Audio variable information.                                                            |
| ○ variableId    | Variable identifier.                                                                   |
| ○ variableName  | Variable name.                                                                         |
| ○ defaultSource | Default audio source.                                                                  |
| • next          | Next column information.                                                               |
| ○ source        | Next source to be applied.                                                             |
| ○ status        | “warning” when the source is different from the one that was set by a Variable Preset. |
| • assigned      | Assigned column information.                                                           |
| ○ source        | Current Source applied.                                                                |

|                |                                                                                                |
|----------------|------------------------------------------------------------------------------------------------|
| ○ status       | “warning” when the source is different from the one that was set by a Variable Preset.         |
| ● nextShow     | Next show column information.                                                                  |
| ○ source       | Next Show Source to be applied.                                                                |
| ○ status       | “warning” when the source is different from the one that was set by a Variable Preset.         |
| ● assignedShow | Assigned show column information.                                                              |
| ○ source       | Current show source applied.                                                                   |
| ○ status       | Returns “warning” when the source is different from the one that was set by a Variable Preset. |
| ● isLocked     | True when the variable has been locked from modifications, false otherwise.                    |

### Example

GET http://<od>/server/floordirector/v1/audio/variables?name=VARIABLE221

```
{
  "meta": {
    "server-date": "2009-04-11T14:22:07,125-05:00",
    "od-host": "srvottoddrv01",
    "od-version": "18.4"
  },
  "statuses": [
    {
      "variable": {
        "variableId": 157,
        "variableName": "VARIABLE221",
        "defaultSource": "Channel 121"
      },
      "next": {
        "source": "Default",
        "status": ""
      },
      "assigned": {
        "source": "Default",
        "status": "warning"
      },
      "nextShow": {
        "source": "None",
        "status": ""
      },
      "assignedShow": {
        "source": "None",
        "status": ""
      },
      "isLocked": false
    }
  ]
}
```

## Subscription

**Socket Only:** It subscribes/unsubscribes the current client connection for receiving autonomous timing updates. The default behavior for new connections is being subscribed.

### Parameters

| URL Parameters | Description          | Values                                                                                    | Required |
|----------------|----------------------|-------------------------------------------------------------------------------------------|----------|
| type           | Type of the request. | <b>optIn</b> – subscribe this connection.<br><b>optOut</b> – unsubscribe this connection. | Yes      |

### Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "meta": {
      "type": "object",
      "properties": {
        "serverDate": {
          "type": "string"
        },
        "odHost": {
          "type": "string"
        },
        "odVersion": {
          "type": "string"
        },
        "correlationId": {
          "type": "string"
        }
      }
    },
    "subscription": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string"
        }
      }
    }
  }
}
```

### Field Description

| Field               | Description                                                                                |
|---------------------|--------------------------------------------------------------------------------------------|
| <b>meta</b>         | Metadata information about the target server.                                              |
| • serverDate        | Format is (ISO8601): YYYY-MM-DD'T'hh:mm:ss[,ddd][['Z']].                                   |
| • odHost            | IP or hostname.                                                                            |
| • odVersion         | If the client sent a correlationId in the request, it would be shown here in the response. |
| • correlationId     | If the client sent a correlationId in the request, it would be shown here in the response. |
| <b>subscription</b> |                                                                                            |
| • response          | Message describing the response.                                                           |

## Example

Socket Request:

```
{
  "protocolVersion": "1",
  "endpoint": "subscription",
  "reqData": "type=optOut",
  "correlationId": "1000001"
}
```

Response:

```
{
  "meta": {
    "serverDate": "2009-04-11T14:22:07,125-0500",
    "odHost": "serverhost01",
    "odVersion": "16.3.0"
  },
  "subscription": {
    "response": "Connection has successfully been removed from the subscriber list."
  }
}
```

## Error Response

In the case of an error is thrown, the server will return an error message.

### Response: JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "error": {
      "type": "string"
    },
    "errorCode": {
      "type": "integer"
    },
    "errorDescription": {
      "type": "string"
    }
  }
}
```

### Field Description

| Field            | Description            |
|------------------|------------------------|
| error            | Error name.            |
| errorCode        | Error code (internal). |
| errorDescription | Detailed description.  |

### Example

```
{
  "error": "Bad Request",
  "errorCode": 400,
  "errorDescription": "Malformed request syntax!"
}
```

### Field Codes

| Code | Description                                                                                                                                                       |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 400  | Bad Request, the server cannot or will not process the request due to an apparent client error (malformed message, invalid protocol version, missing parameters). |
| 404  | Endpoint Not Found, the client requested an invalid endpoint name.                                                                                                |
| 500  | Unknown Error.                                                                                                                                                    |
| 503  | Service Unavailable.                                                                                                                                              |